Project Synopsis

Dept. of ECE, Final year Project 2025-2026

(Even Semester)

TEAM MEMBERS

USN	NAME
1CR23EC069	HARIHARAN G
1CR23EC115	MANJUNATH M

DESIGN AND SYNTHESIS OF AN OUT-OF-ORDER PIPELINED PROCESSOR

1. ABSTRACT

Modern high performance processors exploit Instruction-Level Parallelism(ILP) through Out-of-Order(OOO) execution, allowing instructions to execute as soon as their operands are ready rater than in a strict program order. Despite widespread adoption in industry, OOO micro-architecture remains a largely theoretical topic in most undergraduate curriculum, with few opportunities to experience the complete RTL-to-silicon flow on a realistic design.

This project bridges that gap by designing, verifying, synthesizing, and performing physical design of a complete OOO pipelined processor in SystemVerilog, targeting the Nangate45 a 45 nm standard-cell PDK. The design incorporates a Register Alias Table (RAT)-based renaming unit, a 32-entry Reorder Buffer (ROB), a 16-entry age-ordered Issue Queue with Common Data Bus (CDB) wakeup, and a 64-entry physical register file implemented as a 2-Read 1-Write SRAM macro generated by the OpenRAM open-source memory compiler.

2. INTRODUCTION

The performance of a processor is fundamentally bounded by the number of instructions it can retire per clock cycle. In-order pipelines stall whenever a data hazard or cache miss is encountered, leaving functional units idle. Out-of-Order execution addresses this by dynamically reordering instruction execution: instructions whose operands are available are allowed to proceed even if older instructions are waiting, subject to the constraint that results are committed to the architectural state strictly in program order.

The three mechanisms that make OOO execution possible are: (1) Register Renaming: mapping architectural register names to a larger physical register file, eliminating false WAR and WAW hazards; (2) the Issue Queue: a buffer that holds in-flight instructions and releases them for execution the moment all their source operands have been produced; and (3) the Reorder Buffer: a circular buffer that tracks all in-flight instructions and ensures in-order commit, enabling precise exceptions and branch misprediction recovery.

This project implements these mechanisms in synthesisable SystemVerilog and takes the design through the full ASIC flow: RTL simulation, logic synthesis with Cadence Genus using the Nangate45 PDK, and place-and-route with Cadence Innovus including a hard-block SRAM macro for the physical register

file. The 8-stage pipeline targets 200 MHz, a frequency that is achievable on Nangate45 without aggressive retiming while still presenting meaningful timing challenges across the rename and ROB array paths.

3. LITERATURE SURVEY

3.1. Foundational OOO Algorithm

Tomasulo (1967) introduced the first hardware algorithm for dynamic instruction scheduling in the IBM System/360 Model 91 floating-point unit. His reservation station scheme allows instructions to read operands directly from a common data bus as they become available, eliminating register-name-based hazards without compiler intervention. This work established the conceptual basis for every modern OOO processor.

3.2. Reorder Buffer and Precise Exceptions

Smith and Pleszkun (1985) formalized the concept of the Reorder Buffer, demonstrating that a circular in-order commit queue can decouple the out-of-order execution engine from the need to maintain precise architectural state. Their model which is widely adopted in subsequent designs, is the direct predecessor of the ROB implemented in this project.

3.3. Industrial OOO Implementation

Yeager (1996) - "The MIPS R10000 Superscalar Microprocessor" (IEEE Micro, Vol. 16, No. 2), provides the most detailed publicly available account of a commercial OOO implementation. The R10000 features a 4-wide superscalar OOO pipeline with 64 physical integer registers, a 16-entry integer issue queue, and a 32-entry ROB — parameters directly comparable to this project. Yeager's paper is used as the primary architectural reference.

3.4. Memory Compiler for Register File

Guthaus et al. (2016) - "OpenRAM: An Open-Source Memory Compiler" (ICCAD 2016) — introduced the open-source SRAM compiler used to generate the 64×32b 2R1W register file macro. OpenRAM supports the Nangate45/FreePDK45 technology and produces SPICE netlists, LEF abstracts, and Liberty timing models suitable for direct integration into Cadence Genus and Innovus flows.

4. PROPOSED OUTCOMES

- A fully functional 8-stage OOO processor RTL in SystemVerilog, implementing Fetch, Decode, Register-rename, Dispatch, Issue, Register Read, Execute and Commit stages.
- Timing closure at 200 MHz on Nangate45 with zero Worst Negative Slack (WNS ≥ 0 ns) using Cadence Genus synthesis.
- A placed-and-routed GDS file produced by Cadence Innovus, passing DRC and LVS cleanly.
- A 64×32b 2R1W SRAM macro generated by OpenRAM, integrated as a hard block in the physical design, demonstrating area reduction over a flip-flop-based register file.
- Quantitative post-synthesis and post-route reports: cell count, total area (μm^2), dynamic and leakage power (mW/µW).

5. TOOLS NEEDED

1. Cadence Genus: RTL Synthesis Technology Mapping, timing-driven optimization
2. Cadence Innovus: Floorplanning, Placement, CTS, Routing, GDS Streamout
3. Cadence Incisive: RTL Functional simulation and verification
4. OpenRAM: SRAM macro compiler
5. Nangate45 PDK: Standard-cell .lib / .lef / GDS for synthesis and P&R
6. Python3: OpenRAM configuration and scripting

6. BASE PAPER:

J. Zhao, B. Korpan, A. Gonzales, and K. Asanovic, “ SonicBOOM: The 3rd Generation Berkeley Out-of-Order Machine,” in 4th Workshop on Computer Architecture Research with RISC-V (CARRV), June 2020.

7. REFERENCES

Nangate Inc., "Nangate 45nm Open Cell Library," Nangate OpenCellLibrary, 2011. [Online]. Available: http://www.nangate.com

J. L. Hennessy and D. A. Patterson, Computer Architecture: A Quantitative Approach, 6th ed. Morgan Kaufmann, 2019, ch. 3–4.

M. R. Guthaus, J. E. Stine, S. Ataei, B. Chen, B. Wu, and M. Sarwar, "OpenRAM: An Open-Source Memory Compiler," in Proc. IEEE/ACM Int. Conf. Computer-Aided Design (ICCAD), Nov. 2016, pp. 1–6.

K. C. Yeager, "The MIPS R10000 Superscalar Microprocessor," IEEE Micro, vol. 16, no. 2, pp. 28–41, Apr. 1996. DOI: 10.1109/40.491460.

G. S. Sohi, "Instruction Issue Logic for High-Performance, Interruptable, Multiple Functional Unit, Pipelined Computers," IEEE Transactions on Computers, vol. 39, no. 3, pp. 349–359, Mar. 1990.

J. E. Smith and A. R. Pleszkun, "Implementing Precise Interrupts in Pipelined Processors," IEEE Transactions on Computers, vol. 37, no. 5, pp. 562–573, May 1988.

R. M. Tomasulo, "An Efficient Algorithm for Exploiting Multiple Arithmetic Units," IBM Journal of Research and Development, vol. 11, no. 1, pp. 25–33, Jan. 1967.

Project carried out at CMRIT